KITTI Object Detection: Training, Benchmarking and RAG Dataset Query

An end-to-end pipeline built on the KITTI autonomous driving dataset, covering three phases: training a YOLO model from raw sensor data, benchmarking it across deployment targets, and building a natural language query system for scenario mining and error analysis.

Architecture diagram

Repository Structure

```
preprocessing/download_kittidataset.py   # dataset download
preprocessing/label_convertor.py         # KITTI <-> YOLO label conversion
model/evaluate_yolo.py                   # mAP evaluation
model/benchmark_model.py                 # FP32/FP16/INT8 benchmarking
utils/visualize_predictions.py           # GT vs prediction overlay
queries/generate_faiss_doc.py            # builds FAISS indexes
queries/llmquery_app.py                  # Streamlit query app
queries/fuzzy_rules.json                 # synonym + filter rules for fuzzy
matching
data/                                    # generated indexes, docs, configs
data/datasets/                           # exported YAML dataset manifests
google_collab_training.ipynb             # end-to-end Colab notebook
```

Requirements

```
pip install -r requirements.txt
```

Phase 1: ML Lifecycle

Downloads the KITTI dataset, converts labels between KITTI and YOLO formats, trains a YOLO model on Google Colab, and evaluates with mAP50/mAP50-95. The notebook `google_collab_training.ipynb` sequences all steps end-to-end on Colab GPU.

The pipeline is split between Colab (GPU steps) and local (everything else).

Colab — `google_collab_training.ipynb`:

```
download_kittidataset.py        # fetches images + labels into data/training/
label_convertor.py kitti2yolo   # converts labels, splits into train/val ->
kitti_yolo/
```

```
                                # train YOLO
evaluate_yolo.py                # mAP50, mAP50-95 on val set
visualize_predictions.py        # side-by-side GT vs prediction for one image
benchmark_model.py              # FP32/FP16/INT8 across CPU, GPU, TensorRT
                                # run YOLO predict on data/training/image_2/
                                #   -> saves YOLO-format labels to
runs/detect/predict/labels/
```

Local (run after syncing `runs/` from Google Drive):

```
label_convertor.py yolo2kitti   # converts prediction labels to KITTI format
generate_faiss_doc.py           # builds scene + error FAISS indexes and CLIP
indexes
llmquery_app.py                 # Streamlit query app
```

Phase 2: Model Benchmarking

Benchmarks the trained model across CPU, GPU (PyTorch), and TensorRT at FP32, FP16, and INT8 precision.

Backend	Precision	mAP50	Latency (ms)
CPU	FP32	0.8648	140.42
CPU	INT8	0.8682	717.26
GPU	FP32	0.8648	14.29
GPU	FP16	0.8647	15.16
TensorRT	FP32	0.8682	0.39
TensorRT	FP16	0.8682	0.40
TensorRT	INT8	0.8682	0.40

TensorRT accuracy uses the ONNX model as a proxy — latency is measured from the actual TensorRT engine.

Relevant script: `model/benchmark_model.py`

Phase 3: RAG Query Pipeline

A Streamlit app (`queries/llmquery_app.py`) for querying the KITTI dataset without writing filter code. It has two independent entry points — text-based query and visual CLIP search.

`queries/generate_faiss_doc.py` builds the indexes first: a scene-level FAISS index (object counts, occlusion, truncation per frame), an error-level FAISS index (FP/FN documents with IoU, class, bounding box), and a CLIP image index for visual search. Run this once before launching the app.

Query flow

```
User Query
    |
    +--> Text Query
    |       |
    |       +--> Scene Search  (object counts, occlusion, truncation)
    |       |       |--> fuzzy match? --> filters from rules, LLM skipped
    |       |       |--> LLM (Ollama / Groq) --> structured filters
    |       |       |--> filter match?  --> show matching frames
    |       |       |                       --> Export Dataset (YAML)
    |       |       └--> no match       --> FAISS semantic search --> frames
    |       |                               --> Export Dataset (YAML)
    |       |
    |       └--> Error Analysis  (FP/FN, IoU, class, occlusion)
    |               |--> same LLM / fuzzy / semantic path
    |               └--> results show GT vs prediction overlay
    |
    └--> Visual Search (CLIP)
            |
            +--> Text -> Images   (describe what you want to find)
            |       |--> query expansion via Groq (optional)
            |       |--> CLIP text encode --> FAISS image index search
            |       └--> ViT-B-32 (512-dim, fast) or ViT-L-14 (768-dim, finer)
            |
            └--> Image -> Images  (upload 1-5 example images)
                    |--> CLIP image encode --> mean embedding
                    └--> ViT-B-32 or ViT-L-14 index search
```

Text query details

Both Scene Search and Error Analysis go through the same resolution path. First, the query is checked against `fuzzy_rules.json` — terms like "crowded", "few cyclists", "heavy occlusion" are pre-mapped to exact numeric filters with synonyms. If the query is fully covered by fuzzy rules (after stripping stopwords with no filter intent), the LLM is skipped entirely and filters are applied directly. The UI shows "fuzzy rules only — LLM skipped" in this case.

If fuzzy rules don't cover everything, the query goes to the LLM. Two backends are supported and switchable at runtime from the sidebar: Ollama (local, private, no API key) and Groq (cloud, ~10x faster). Both run at temperature=0 for deterministic output. The LLM returns structured filters which are applied against the FAISS index. If nothing matches, it falls back to sentence-transformer semantic search over the same index.

Error Analysis results include a side-by-side overlay: GT boxes in green, predictions in yellow, false positives in red, false negatives in blue.

Dataset export

After any query returns results, an "Export Dataset (YAML)" button appears below the frames. Clicking it writes a manifest to `data/datasets/` that records which frames matched, the total match count, and the LLM filter output that produced them.

```yaml
query: more than 5 pedestrians
mode: Scene Search
exported_at: '2024-06-30T14:22:01'
total_matched: 209
exported: 5
llm_output:
  filters:
    num_pedestrians:
      '>': 5
  semantic_query: scenes with many pedestrians
frames:
- id: '002445'
  image: data/training/image_2/002445.png
  label: data/training/label_2/002445.txt
```

This demonstrates the data lifecycle — natural language query to a reproducible, traceable frame subset — without coupling to a specific annotation tool.

Visual search details

Text-to-image search encodes a text description directly with CLIP and finds the most visually similar frames. When Groq is active, the query is first expanded into a richer visual description before encoding — this helps with short queries like "pedestrian crossing sign" where the raw text doesn't encode enough visual detail.

Image-to-image search takes one to five uploaded images, encodes each with CLIP, averages the embeddings, and retrieves the most similar frames. Multiple images make the query more robust.

Both modes support two CLIP models selectable from the sidebar. ViT-B-32 is faster and works well for scene-level queries. ViT-L-14 uses 14x14 pixel patches instead of 32x32, giving finer spatial resolution — better for small objects like signs but slower.

Example queries

```
# Scene Search
"more than 5 pedestrians"
"busy intersection with few cyclists"      <- fuzzy fast path, LLM skipped
"heavy occlusion and more than 2 cars"

# Error Analysis
"false positives for cyclists"
"missed pedestrians with occlusion 3"
"IoU < 0.4 errors for cars"

# Visual Search (Text -> Images)
"pedestrian crossing sign"
"road with tram tracks"
"construction worker near vehicle"
```

Running the app

```
ollama run llama3                                    # skip if using Groq
cd queries
python -m streamlit run llmquery_app.py
```

TensorRT requires the NVIDIA TensorRT SDK installed separately. For the query app, either Ollama running locally or a Groq API key (free at console.groq.com).